

TECHNICAL INTERVIEW PREPARATION GUIDE

Tailored Practice Questions & Detailed Scenario-Based Answers for Harshana Ayanki

Target Role: Senior Data Engineer (5+ Years Experience)

Key Skill Areas Covered: Apache Spark & Databricks Optimization, GCP & Azure Pipeline Architecture, Workflow Orchestration (Airflow), Data Security (PGP) & Quality Controls, and Real-time Streaming/JSON Processing.

1. APACHE SPARK & DATABRICKS PERFORMANCE TUNING

DATABRICKS & SPARK OPTIMIZATION

Q1: How did you diagnose and achieve a 96% execution time reduction on the Databricks Spark Bybox report? What specific techniques did you apply?

Answer: Diagnosing Spark performance bottlenecks requires systematically analyzing execution plans and I/O metrics. By reviewing the Spark UI (DAG Visualization, Event Timeline, and SQL tab), I identified severe input file scanning overhead, data skewness, and the "small file problem" caused by over-partitioning.

Applied Solution & Example:

- **Predicate Pushdown & Partition Pruning:** Pushed filter conditions down to the storage level so Spark reads only relevant partition folders rather than full scans.
- **Z-Ordering on Delta Tables:** Colocated related data along specified columns (e.g., `OPTIMIZE bybox\_table ZORDER BY (product\_id, date)`), significantly reducing the volume of data scanned via file skipping.
- **Auto-Tuning Delta Lake:** Enabled Databricks automatic file compaction (`optimizedWrites` and `autoCompact`) to resolve small file fragmentation.

```
%sql
-- Compact small files and layout data using Z-Order
OPTIMIZE fin_retail.bybox_report
ZORDER BY (account_id, transaction_date);
```

DELTA LAKE ARCHITECTURE

Q2: How do you address the "Small File Problem" in Delta Lake pipelines, and what causes over-partitioning?

Answer: Over-partitioning occurs when data is partitioned on high-cardinality columns (e.g., timestamp or customer ID), resulting in thousands of tiny files (a few KB each). Spark overhead increases drastically due to metadata management and listing operations.

Example Strategy: Partition tables only on low-cardinality columns with large data volumes (e.g., `YYYY-MM`). For small file management, combine manual `OPTIMIZE` runs with Databricks Auto-Tuning properties:

```
SET spark.databricks.delta.properties.defaults.autoOptimize.optimizeWrite = true;
SET spark.databricks.delta.properties.defaults.autoOptimize.autoCompact = true;
```

2. CLOUD ARCHITECTURES (GCP & AZURE) & WORKFLOW DESIGN

GCP DATA ENGINEERING

Q3: At Accenture, how did you choose between Cloud Dataflow and Dataproc for financial data processing?

Answer: The choice depends on processing paradigm (streaming vs. batch), framework portability, and cluster management needs:

- **Cloud Dataproc:** Chosen for existing Apache Spark/PySpark workloads that require open-source ecosystem compatibility and fine-grained cluster configurations.
- **Cloud Dataflow:** Chosen for unified stream/batch pipelines using Apache Beam, offering serverless auto-scaling without cluster management overhead.

Example Scenario: For batch financial reconciliations with legacy Spark code, Dataproc ephemeral clusters were spun up on-demand via Airflow's `DataprocCreateClusterOperator`, processed datasets, and auto-deleted upon task completion to minimize cloud spend.

AZURE & INGESTION

Q4: How did you design the ingestion pipeline for Openfleet highly nested JSON streaming data into Azure Synapse?

Answer: Streaming nested JSON requires a multi-tier Bronze-to-Gold (Medallion) Lakehouse design:

1. **Bronze (Raw Storage):** Openfleet MuleSoft push events ingested into Azure Data Lake Storage Gen2 (ADLS Gen2) in raw JSON format using Azure Data Factory / Event Hubs.
2. **Silver (Flattening & Cleansing):** PySpark on Azure Databricks reads raw JSON using `from\_json` and schema definitions, flattens arrays/structs, and validates schema consistency.
3. **Gold (Synapse Warehouse):** Aggregated and curated datasets loaded into Azure Synapse Analytics tables for business reporting.

```
# PySpark Nested JSON Flattening Example
from pyspark.sql.functions import col, explode

df_flattened = raw_json_df.select(
    col("payload.header.transaction_id").alias("tx_id"),
    explode(col("payload.details.line_items")).alias("item")
).select("tx_id", "item.item_id", "item.price")
```

3. AIRFLOW ORCHESTRATION, SECURITY & SLA MANAGEMENT

APACHE AIRFLOW

Q5: How do you handle pipeline dependency management and failure alerts in Apache Airflow for critical BFS banking datasets?

Answer: Banking datasets demand strict data SLAs, idempotency, and immediate alerting.

Implementation Strategy:

- **Retries & Timeouts:** Configured DAG ``default_args`` with ``retries=3``, exponential ``retry_delay``, and ``execution_timeout``.
- **SLA Monitoring:** Used Airflow ``sla_miss_callback`` and ``on_failure_callback`` to post instant notifications to PagerDuty/Teams/ServiceNow.
- **Sensors & Trigger Rules:** Used ``ExternalTaskSensor`` to wait for dependent upstream financial DAGs (e.g., ledger balances) before executing regulatory export tasks.

DATA SECURITY & COMPLIANCE

Q6: How did you implement PGP encryption for regulatory reporting on GCP to maintain 100% security compliance?

Answer: Federal and banking regulations require sensitive datasets to be encrypted at rest and in transit prior to external transmission.

Execution Workflow:

1. Extracted financial datasets from BigQuery/Dataprocs to Cloud Storage (GCS).
2. Triggered a serverless Cloud Function or PySpark PGP module using GNU Privacy Guard (``gnupg``) with recipient public keys stored in Secret Manager.
3. Transmitted encrypted files (``*.pgp`/`*.gpg``) securely via SFTP to federal compliance bodies, keeping private keys isolated.

OPERATIONAL EFFICIENCY

Q7: How did you achieve an 80% reduction in ADF pipeline execution time using an archival strategy?

Answer: Scans on growing transactional tables degrade pipeline efficiency over time.

Example Approach: Partitioned hot transactional data (current year) from cold historical data. Engineered an ADF pipeline that automatically moves records older than 12 months to compressed Parquet files in cold storage (ADLS Gen2/GCS Archive). Source queries then ran strictly against light hot tables, slashing ADF lookup and execution times by 80%.